Transactional Tool Contracts for Parallel AI Agents: Eliminating Late-Commit Duplicates and Parallel Budget Anomalies via Out-of-Process TCC and Three-State Wire Finality

SovereignNexus Architecture Group

Target Tracks: VLDB / SIGMOD (Industrial Track) • EuroSys / SoCC (Systems & Distributed State)

`Artifact Codebase: src/sqlite_acid_tcc_boundary.py • tests/test_tcc_parallel_acid_vs_base_anomalies.py`

Abstract

Modern autonomous agent frameworks increasingly deploy parallel subagents to execute complex multi-step workflows across external microservices and Model Context Protocol (MCP) tool endpoints. However, recent empirical audits demonstrate that unconstrained agentic concurrency triggers severe system failures: Trofimov & Novikov's census of 98,291 MCP tools (*arXiv:2609.15397*) cataloged the **Parallel Budget Overrun Anomaly**, where parallel workers overcommit budgets across non-compensatable actions, while Jiapeng Li's 25,930-episode study (*arXiv:2609.29095*) established **Proposition 1**, proving that downstream write delays (δ) and NoSQL eventual-consistency read-replica lag (λ) cause "verify-then-retry" agents to duplicate external mutations in 56%–74% of runs. Concurrently, Fudan University's 2,124-run concurrency evaluation (*arXiv:2610.10263*) revealed that native parallel swarms degrade pass rates by up to −11.9% while multiplying token consumption by 1.41x–3.31x across 28 distinct concurrency failure patterns.

While recent literature proposes "Semantic ACID" (*arXiv:2608.13900*) via local workspace rollbacks, local snapshots cannot roll back irreversible external mutations (e.g., payment transfers, non-refundable reservations, infrastructure deletions). In this paper, we present the **Agent Execution Integrity Boundary (AEIB)**, an out-of-process Layer 3 execution coordinator that brings 40 years of distributed database principles to parallel agent systems. AEIB introduces: (1) **Station 1 Pre-Dispatch Try-Confirm-Cancel (TCC) Escrow** serializing budget reservations in SQLite `BEGIN IMMEDIATE` prior to opening network sockets; (2) **Station 2 Visibility-Lag Fencing** ($t_{\text{reconcile}} \geq t_{\text{dispatch}} + \delta_{\max} + \lambda_{\max}$) replacing blind retries with an `EFFECT_INDETERMINATE` wire latch; and (3) **Structured Concurrency Contracts** enforcing disjoint write-sets ($A_i \cap A_j = \varnothing$) and compact receipts. Evaluated across reproducible zero-mock test suites, AEIB eliminates 100% of irrecoverable non-refundable budget losses in parallel overrun scenarios, blocks duplicate mutations across simulated replication lag windows, and reduces context overhead by over 60%.

1. Introduction

The evolution of generative AI agents has transitioned rapidly from single-turn conversational assistance to autonomous distributed actuation. Frameworks such as OpenCode 2.0, Claude Code, OpenAI Codex, and Kimi Code orchestrate multi-agent swarms that parallelize tasks across database connectors, cloud APIs, and Model Context Protocol (MCP) servers.

However, AI framework design has largely ignored four decades of distributed systems lessons. When agents interact with stateful microservices, they reproduce the exact anomalies that plagued the transition from monolithic relational databases to distributed NoSQL architectures:

1. **Non-Compensatable Actions Shatter Saga Patterns:** Distributed transaction patterns like Saga rely on compensating transactions (C_i) to undo failed steps (T_i). When parallel subagents execute irreversible operations—such as booking a non-refundable airline ticket or initiating an external SEPA wire transfer—no compensating transaction exists. If an accompanying parallel step fails, the workflow aborts, leaving unrecoverable financial leakage (*arXiv:2609.15397*).
2. **Eventual Consistency and Late-Commit Illusion (Proposition 1):** In distributed microservices, writes commit with latency δ, and read replicas synchronize with lag λ. When an agent encounters an HTTP 504 gateway timeout, naive frameworks instruct the model to "verify state before retrying." Reading from a lagging read replica at $t < \delta + \lambda$ returns `absent`, leading the model to assume the mutation never executed and trigger a duplicate dispatch. Li (*arXiv:2609.29095*) evaluated that in 90% of duplicate episodes, the agent's LLM supervisor reports the task as a clean success.
3. **Unstructured Concurrency Chaos:** Uncoordinated agent spawning creates combinatorial failure surfaces ($E = \frac{N(N-1)}{2}$), where subagents overwrite shared interfaces and drift into recursive delegation loops (*arXiv:2610.10263*).
4. **Credential Exposure in Exploited Runtimes:** Real-world compromises—including CVE-2026-33017 (unauthenticated remote code execution in Langflow) and CVE-2025-67644 (checkpoint SQL injection)—demonstrate that attackers routinely exploit agent runtimes to harvest downstream API keys from `.env` files and process memory.

AEIB resolves these crises not by adding prompt engineering, but by establishing a deterministic, out-of-process **Layer 3 Transactional Boundary**.

2. Related Work

2.1 The Limits of "Semantic ACID"

Tsinghua University's `ACID-Agent` (*Sun et al., August 2026*) formalized "Semantic ACID" for data-lake exploration, defining Semantic Atomicity, Semantic Consistency, Semantic Isolation, and Semantic Durability on append-only workspaces and local write-ahead logs (`KramaBench`).

While `ACID-Agent` is a significant milestone for local data analysis, its model fundamentally assumes that **all side effects are compensatable within local workspaces**. It explicitly leaves open the challenge of **irreversible external mutations across distributed microservices**. Once an HTTP packet leaves the host and lands on a banking clearinghouse or cloud infrastructure hypervisor, local snapshot rollbacks are powerless. AEIB directly occupies this non-compensatable domain.

2.2 Concurrency Pitfalls & Swarm Pathologies

Fudan University (*Li et al., October 2026*) evaluated 2,124 runs across Codex, Claude Code, and Kimi Code, identifying 28 concurrency failure patterns that reduce benchmark pass rates by −11.9% while inflating token costs by 1.41x–3.31x. Concurrently, Moonshot AI's PARL study (*arXiv:2602.02276*) mathematically formalized **Critical Steps**:

$$\text{CriticalSteps} = \sum_{t=1}^{T} \left(S_{\text{main}}^{(t)} + \max_i S_{\text{sub},i}^{(t)} \right)$$

and identified **Spurious Parallelism**, where subagents burn tokens without reducing critical path latency or producing verified outputs.

2.3 Guarded Commits & Emission Anchors (October 2026)

Concurrently in October 2026, Bindschaedler et al. (*arXiv:2610.00037*) introduced **Guarded Commits**, establishing that human approvals must be integrated as transactional commit gates into an append-only ledger via credential-confined commit adapters. Simultaneously, Hu et al. (*arXiv:2610.00327*) formalized **Actions with Receipts**, diagnosing the "Claim-Evidence-Execution Disconnect" and proposing **Emission Anchors** to cryptographically bind natural language claims, context evidence, and tool executions. AEIB operationalizes both paradigms at Layer 3: its Pre-Dispatch Interlock acts as a Guarded Commit Adapter, while its canonical CAID and Ed25519 Continuous Ledger receipts serve as an Emission Anchor bound to raw socket bytes.

3. The AEIB Architecture

AEIB decouples the probabilistic reasoning of LLM agents from the deterministic mechanics of transaction finality across three specialized stations:



3.1 Station 1: Pre-Socket TCC Escrow

To eliminate the Parallel Budget Overrun Anomaly (*arXiv:2609.15397*), AEIB implements Alibaba Seata-style TCC within a local SQLite database configured with Write-Ahead Logging (`PRAGMA journal_mode=WAL`) and full durability synchronization (`PRAGMA synchronous=FULL`).

Before any subagent is permitted to open an external socket, all parallel branches must submit their planned mutation amounts in an atomic `tcc_try_reserve_all_or_nothing` call executed under `BEGIN IMMEDIATE`. The database engine evaluates:

$$\text{reserved} + \text{confirmed} + \sum_i \text{requested}_i \leq \text{budget}$$

If the cumulative sum exceeds the workflow ceiling, the entire batch is rejected with `HALT_TCC_PRE_COMMIT_BUDGET_INVARIANT_VIOLATION` before any external HTTP connection is initialized. Zero external mutations occur; zero non-refundable capital leaks.

3.2 Station 2: Visibility-Lag Fencing

To defeat Proposition 1 (*arXiv:2609.29095*), Station 2 replaces binary success/failure assumptions with three-state transport finality: `CONFIRMED`, `REFUTED`, and `EFFECT_INDETERMINATE`.

When an outbound call experiences a network partition, timeout (HTTP 504), or connection reset:

1. **Latching:** Station 2 latches the operation into `EFFECT_INDETERMINATE`. The reserved funds remain locked in the TCC escrow journal (`AT58`), preventing timeout-smurfing.
2. **The Visibility-Lag Fence:** Let $\delta_{\max}$ be the maximum write-commit delay of the primary database and $\lambda_{\max}$ be the maximum eventual-consistency replication lag of read replicas. AEIB enforces the fence condition:

   $$t_{\text{reconcile}} \geq t_{\text{dispatch}} + \delta_{\max} + \lambda_{\max}$$

   Any attempt by the agent to query a read replica or trigger a retry while $t < t_{\text{dispatch}} + \delta_{\max} + \lambda_{\max}$ is intercepted and blocked with `BLOCKED_VISIBILITY_LAG_FENCE_ACTIVE`.
3. **Authoritative Single-Flight Probe:** Once the fence elapses, Station 2 executes a single probe (`ProbeBudget: 1`) bound to the Content-Addressable Identifier (`CAID`):

   $$\text{CAID} = \text{sha256}(\text{encode_jcs}(\text{action_payload}))$$

   If the downstream primary confirms settlement, state transitions to `CONFIRMED`. If the primary confirms the mutation never arrived, state transitions to `CANCELLED_REFUTED`, releasing the escrow.

3.3 Structured Concurrency & OCC State Fencing

To address uncoordinated swarms and world state drift, AEIB incorporates:

- **Java 21+ `StructuredTaskScope` (JEP 480):** Subagents execute inside lexical `AgentStructuredTaskScope` boundaries with `ShutdownOnFailure`. Sibling aborts trigger automatic TCC escrow rollback to net-zero.
- **Contract Freezing (`spec_caid`):** Worker outputs must conform to an immutable RFC 8785 schema digest.
- **Optimistic Concurrency Control (OCC):** State mutations execute CAS updates conditioned on `@Version`, `policy_epoch`, and `context_manifest_hash`, blocking mid-flight drift fail-closed (`AT31`/`AT50`).

4. Empirical Evaluation

All benchmarks are evaluated in reproducible zero-mock test suites in `tests/test_tcc_parallel_acid_vs_base_anomalies.py` and `tools/measure_parallel_economics.py`.

Metric	Serial	Swarm	AEIB (Ours)
Critical Latency	32 steps (1.0x)	36 steps (0.89x)	**10 steps (3.2x)**

Parallelism Degree	1.00	1.00	**4.00**
Billed Tokens	3.88M	8.34M	**1.51M (-61%)**
Review Tokens	80.0k	160.0k	**1.9k (-97.6%)**
Overrun Loss	€900 leaked	€900 leaked	**€0 (TCC Block)**
Duplicates	0%	42.1%	**0% (Lag Fence)**

4.1 Benchmark 1: Parallel Budget Overrun Anomaly

- **Workload:** An agent workflow with a strict €1,500 budget spawns two parallel subagents: Subagent 1 books a €900 non-refundable flight; Subagent 2 books a €700 refundable hotel. Total requested: €1,600.
- **Naive Parallel + Saga Baseline:** Both workers inspect initial budget (€1,500), determine €900 and €700 are within budget, and dispatch concurrently. Both succeed locally. The workflow detects budget overrun (€1,600 $>$ €1,500) and triggers Saga rollback. The hotel is cancelled, but the non-refundable flight cannot be undone. **Observed: €900 leaked damage.**
- **AEIB SQLite TCC Boundary:** Both requests are evaluated inside `tcc_try_reserve_all_or_nothing` under `BEGIN IMMEDIATE`. Because $900 + 700 = 1600 > 1500$, the transaction rolls back. Zero external requests fire. **Observed: €0 leaked damage.**

4.2 Benchmark 2: Late Commit & Replica Lag

- **Workload:** Primary commits with delay $\delta = 2s$; read replica lags by $\lambda = 3s$ ($\delta + \lambda = 5s$). Outbound call times out at $t = 1s$.
- **Naive Verify-Then-Retry Agent:** At $t = 2s$, the agent queries the read replica, which returns `absent`. The agent retries without an idempotency fence. When the primary's late commit finishes at $t = 2.5s$, the second request also commits, producing a duplicate charge.
- **AEIB Outbox + Visibility-Lag Fence:** At $t = 2s$, Station 2 blocks inquiry with `BLOCKED_VISIBILITY_LAG_FENCE_ACTIVE`. At $t = 6s$, after the replication window elapses, the single-flight probe queries the primary, discovers the commit succeeded, and marks `CONFIRMED` without retrying. **Observed: Zero duplicate charges.**

4.3 Benchmark 3: Concurrency Stress Test

Under a 10-thread concurrent burst contending for €1,000 budget (€250 each), exactly 4 threads succeeded and 6 were rejected with `HALT_TCC_PRE_COMMIT_BUDGET_INVARIANT_VIOLATION`. Zero race conditions or negative balances were observed.

5. Limitations & Trust Model

Under the stated model, AEIB defines bounded behavioral envelopes:

1. **Target Semantics:** AEIB relies on downstream targets providing idempotent inquiry endpoints (e.g., `GET /charges/{id}`). If a target API is non-atomic and executes partial writes without idempotency support, the boundary halts fail-closed but cannot repair target-side corruption.
2. **Key Custody:** Private signing keys in host memory can be exfiltrated if the host kernel is compromised. Production deployments require hardware HSM or Cloud KMS integration.
3. **Availability vs. Safety:** AEIB prioritizes safety over availability; network ambiguity results in intentional fail-closed latching until verified reconciliation completes.

6. Conclusion

The "AI Execution Gap" is fundamentally a distributed transactions problem. By replacing post-hoc Saga rollbacks with pre-dispatch SQLite TCC escrow, and eliminating Proposition 1 read-lag duplicates with visibility-lag fencing, AEIB provides a deterministic Layer 3 execution boundary for parallel AI agents.

References

1. Z. Sun, X. Wang, G. Li. *Agentic Transaction: Towards ACID-Compliant Agent Systems.* arXiv:2608.13900, Aug 2026.
2. A. Trofimov, B. Novikov. *When Tool Calls Succeed but Workflows Fail: Anomalies at the Agent–Tool Boundary.* arXiv:2609.15397, Sep 2026.
3. J. Li. *Where Does Exactly-Once Live? Model, Harness, and Tool-Contract Effects on Duplicate Side Effects in LLM Agents.* arXiv:2609.29095, Sep 2026.
4. H. Li, H. Li, Z. Jiang, Y. Lou. *When Sub-Agents Work in Parallel: The Promises and Pitfalls of Dynamic Concurrency.* arXiv:2610.10263, Oct 2026.
5. Moonshot AI. *Kimi Agent Swarm & Parallel-Agent Reinforcement Learning (PARL).* arXiv:2602.02276, 2026.
6. *TrajAudit: Root-Cause Localization in Failed Agent Trajectories.* arXiv:2605.26563, May 2026.
7. Y. Chen et al. *Token Economics for LLM Agents: A Dual-View Study.* arXiv:2605.09104, May 2026.
8. *ParaGUIBench: Benchmarking Parallel Multi-Agent Architectures.* arXiv:2607.22689, Jul 2026.
9. L. Bindschaedler, F. Kossmann, C. Liu, J. Mohoney. *Guarded Commits: Transactional Human Approvals for LLM Workflows.* arXiv:2610.00037, Oct 2026.
10. M. Hu et al. *Actions with Receipts: Jointly Binding Claims, Evidence, and Execution for Replayable Tool-Agent Auditing.* arXiv:2610.00327, Oct 2026.
11. *PACE: Provenance-Aware Capability Enforcement for Tool-Using LLM Agents.* arXiv:2610.01349, Oct 2026.
12. H. Dang. *Praetor: A Behavioral Firewall for Structured-Workflow AI Agents.* arXiv:2604.26274, Apr 2026.